

PROJECT REPORT

Automation of an E-Commerce Web Application Using Selenium WebDriver with Python

Capstone Project 1

***Name: Ahana Bandhu, Enrollment No.: 12023002022120, Stream: CST
Institute of Engineering and Management, Kolkata***

Email Id: ahanadgp2003@gmail.com

Github Link: <https://github.com/AhanaBandhu/Capstone-Project-1-Automate-an-Web-Application-Using-Selenium-WebDriver-with-Python>

Video Demonstration Link:

<https://drive.google.com/file/d/1AUknmOfgIVlgE7nL3TIUxPnU6T6F1QBo/view?usp=sharing>

Field	Detail
Application Under Test	https://tutorialsninja.com/demo/ (OpenCart demo)
Automation Tool	Selenium WebDriver 4.x
Programming Language	Python 3.x
Design Pattern	Page Object Model (POM)
Test Framework	unittest
Date	September 2026

1.Capstone Assignment1: Automate an Web Application Using Selenium WebDriver with Python. Business Scenario

A customer wants to purchase a product from an E-Commerce site.

The automation should:

1. Launch browser
2. Login to application
3. Search product
4. Add product to cart
5. Update quantity
6. Verify cart details
7. Capture screenshots
8. Read test data from Excel/JSON
9. Handle popup/alerts if available
10. Generate execution report

Application

Use any public demo application:

- <https://tutorialsninja.com/demo/> OR
- <https://demo.opencart.com/> OR
- <https://automationexercise.com/>

1. Introduction

This report documents the design, implementation and execution of an automated test suite that simulates a real customer purchase journey on a public e-commerce demo application. The suite is built with Selenium WebDriver and Python, following the Page Object Model (POM) design pattern to keep the automation maintainable, readable and reusable.

The objective of this capstone assignment is to demonstrate practical, industry-aligned automation skills: browser automation, page abstraction, external test-data management, alert/popup handling, evidence capture (screenshots) and structured execution reporting.

1.1 Business Scenario

A customer visits an e-commerce website with the intent to purchase a product. The automation reproduces this journey end-to-end and verifies that the system behaves correctly at each step of the transaction.

1.2 Objectives

1. Launch a browser and navigate to the application under test.
2. Authenticate into the application (login, with automatic account registration as a fallback).
3. Search for a product using the site's search functionality.
4. Add the located product to the shopping cart.

5. Update the quantity of the item within the cart.
6. Verify that the cart reflects the correct product and quantity.
7. Capture screenshots as evidence at every functional step.
8. Read test data from an external JSON (or optional Excel) file rather than hardcoding it.
9. Detect and gracefully handle any JavaScript alerts or in-page popups.
10. Generate a structured HTML execution report summarizing the run.

2. Tools and Technologies

Category	Technology / Tool
Programming Language	Python 3.x
Automation Library	Selenium WebDriver 4.x (Selenium Manager auto-resolves the browser driver)
Test Framework / Runner	Python unittest
Design Pattern	Page Object Model (POM)
Test Data Formats	JSON (primary) and Excel via openpyxl (optional, auto-detected)
Reporting	Custom self-contained HTML report generator
Browser	Google Chrome (Firefox also supported via config)
Application Under Test	tutorialsninja.com/demo — a public OpenCart demo store

3. Project Architecture

The framework separates concerns into four layers, which keeps test logic decoupled from both the UI locators and the low-level browser/reporting mechanics:

Layer	Responsibility	Location
Configuration	URLs, timeouts, browser/headless settings	config/config.py
Page Objects	Locators and page-specific actions only	pages/
Utilities	Driver creation, data reading, screenshots, alerts, reporting	utils/
Test Layer	Orchestrates the end-to-end business flow and assertions	tests/test_ecommerce_flow.py

This layering means a UI change on the site only requires updating the relevant Page Object's locators, never the test logic itself — and the test logic reads as a plain business narrative (login → search → add to cart → update quantity → verify).

4. Project File Structure

ecommerce_automation/	
├─ main.py	Entry point - runs the full suite
├─ requirements.txt	Python dependencies

— README.md	Setup and usage instructions
— config/	
— config.py	URLs, timeouts, browser settings
— data/	
— testdata.json	Test data: credentials, product, qty
— testdata.xlsx.md	Notes for the optional Excel source
— pages/	Page Object Model
— base_page.py	Shared explicit-wait helpers
— login_page.py	Login + Register page object
— search_page.py	Product search page object
— cart_page.py	Shopping cart page object
— utils/	
— driver_factory.py	WebDriver creation (Chrome/Firefox)
— data_reader.py	JSON / Excel test-data reader
— screenshot_utils.py	Screenshot capture helper
— alert_handler.py	JS alert & in-page popup handling
— report_generator.py	HTML execution report builder
— tests/	
— test_ecommerce_flow.py	unittest case - the full flow
— reports/	
— screenshots/	Populated automatically at run time
— execution_report.html	Generated automatically at run time

5. Test Data Management

All test data is externalized so the suite can be re-run with different inputs without touching code, satisfying requirement 8 of the assignment.

5.1 Primary source — JSON

data/testdata.json holds the registration/login template, the product search term, and the initial and updated cart quantities. An {timestamp} placeholder in the email field is resolved at runtime so every execution registers a unique account on the demo store.

5.2 Optional source — Excel

utils/data_reader.py checks for data/testdata.xlsx first. If present, it reads a single-row 'TestData' sheet via openpyxl and maps its columns onto the same internal structure used by the JSON path — so the rest of the framework is completely agnostic to which source supplied the data.

6. Automated Execution Flow

The test in tests/test_ecommerce_flow.py executes the following steps in sequence. Every step is wrapped in a helper that records a PASS/FAIL status, a timestamp, and a screenshot to the report — so a failure at any step is captured with evidence rather than silently aborting.

#	Step	Key Actions
1	Launch Browser / Application	Chrome session started via Selenium Manager; base URL opened; any startup alert handled
2	Login to Application	Attempts login with configured credentials; auto-registers a new account if the login fails (first run)
3	Search Product	Enters the configured search term in the header search box and submits

4	Add Product to Cart	Clicks 'Add to Cart' on the first matching result; waits for the success banner
5	Update Quantity	Opens the cart page, updates the quantity field, and clicks the update/refresh control
6	Verify Cart Details	Asserts the product name and quantity shown in the cart match what was added/updated
7	Screenshot Capture	A screenshot is captured after every step (success and failure) throughout the run
8	Read Test Data	All of the above is parameterized from data/testdata.json (or testdata.xlsx)
9	Alert/Popup Handling	utils/alert_handler.py checks for and dismisses native JS alerts and in-page banners
10	Generate Report	utils/report_generator.py compiles all step results into execution_report.html

7. Alert and Popup Handling

utils/alert_handler.py provides two mechanisms:

- `handle_js_alert_if_present()` — waits briefly for a native browser alert/confirm/prompt dialog and accepts (or dismisses) it if one appears, without failing the test when none does.
- `dismiss_inpage_popup_if_present()` — closes a Bootstrap-style in-page modal (e.g. a cookie-consent banner or newsletter popup) identified by a CSS close-button selector, if present on the page.

Both are non-blocking: if no alert or popup appears, execution simply continues, which keeps the suite resilient to the demo site's inconsistent popup behavior.

8. Execution Reporting

utils/report_generator.py produces a single self-contained HTML file (reports/execution_report.html) with no external report-library dependency. The report includes:

- An overall PASS/FAIL result and a summary of total, passed and failed steps
- Run start time, end time and total duration
- A row per step, with timestamp, status badge, details, and a clickable screenshot thumbnail

This design keeps the report portable — it can be opened directly in a browser or attached to an email/build artifact without any server or plugin.

9. How to Run the Suite

```
# 1. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Run the suite
python main.py
```

```
# or
python -m unittest tests/test_ecommerce_flow.py -v
```

Chrome runs in normal (headed) mode by default; set `HEADLESS=true` as an environment variable to run headless, e.g. for CI pipelines.

10. Challenges and Design Decisions

Challenge	Resolution
The demo site has no fixed public login credentials	<code>login_or_register()</code> attempts login first and transparently falls back to registering a fresh throwaway account, so the suite is runnable out-of-the-box
Cart totals update via AJAX after a quantity change	A short explicit wait follows the update-button click before re-reading the row values
Driver/browser version drift	Selenium 4's built-in Selenium Manager resolves the correct driver automatically — no manual chromedriver maintenance
Keeping the report readable without extra dependencies	A hand-rolled, single-file HTML report was used instead of pulling in a third-party reporting library

11. Conclusion

The delivered framework satisfies all ten requirements of the capstone assignment: it launches the browser, authenticates, searches, adds to cart, updates quantity, verifies the cart, captures screenshots at every step, is fully data-driven from an external JSON/Excel source, tolerantly handles alerts/popups, and produces a structured HTML execution report. The Page Object Model keeps the codebase maintainable and ready to extend with further scenarios (e.g. full checkout, order history, multi-product carts).

12. Appendix — Source Code

The complete, runnable source code (all modules listed in Section 4) accompanies this report as a separate submission package: `ecommerce_automation.zip`. Key files to review first: `tests/test_ecommerce_flow.py` (the orchestration logic) and `pages/*.py` (the Page Object locators and actions).